Elasticsearch

認識如何用 Elasticsearch 解決系統設計裡常見的搜尋與檢索問題。

許多系統設計題目都會牽涉「搜尋與檢索」：我有一大堆「東西」，想快速找到對的那一個或那一批。一般資料庫在這方面其實不差（例如帶 full-text index 的 Postgres 就夠應付不少情境），但當規模或需求變複雜時，你會想用專門的搜尋系統，而這類需求往往還伴隨排序、篩選、排名、分面（faceting）等。這時就會提到最常被拿來用的搜尋引擎之一：Elasticsearch。

從面試角度，這份講義會從兩個方向幫你理解 Elasticsearch：

第一，學會怎麼「用」Elasticsearch。這會成為你工具箱裡很實用的一項。搜尋與檢索類的題目很少會複雜到 Elasticsearch 做不了。如果你面的是新創或偏產品架構的職位，懂 Elasticsearch 很有幫助。若你已經用過，可以跳過這部分。

第二，理解 Elasticsearch「底層是怎麼運作的」。作為一個典型的分散式系統實作，Elasticsearch 整合了很多概念，這些概念即使不搞搜尋也很有用。有些面試官（不只我）可能會假設 Elasticsearch 不存在，要你從頭解釋一些高階概念，這在偏基礎設施、尤其是雲端公司的職位比較常見。

Elasticsearch 是做了十幾年的大專案，功能非常多，這裡無法全部涵蓋，但會盡量把重要部分講清楚。我們開始。

基本概念

先從名詞開始。從使用端來看，Elasticsearch 裡重要的概念是：document（文件）、index（索引）、mapping（映射）和 field（欄位）。

核心概念

Document（文件）

Document 是你實際在搜尋的「一筆資料」單位。「文件」不一定是網頁或部落格文章，不必被字面意思限制，就當成任意一個 JSON 物件。例如書店裡的書可以長這樣：

buildmoat.org

```
{
  "id": "abc123",
  "title": "BuildMoat - Intensive Training Camp (System Desig
  "author": "Terry & Bohr",
  "price": 2000,
  "createdAt": "2026-03-15T00:00:00.000Z"
}
```

Index（索引）

Index 是一群 document 的集合。每個 document 都有一個唯一 ID 以及一組 field（鍵值對），也就是你要搜尋的資料。可以把 index 想成資料庫的 table。搜尋是針對這些 index 做的，回傳符合條件的 document。

要注意這裡的「index」和一般說的「索引」（那種讓查詢變快的輔助資料結構）是不同概念，後面會盡量用上下文區分，避免混淆。

我們可以為書本建一個 index，也可以為評論、使用者、訂單等各種業務實體各自建 index。

Mapping 與 Field

Mapping 是 index 的 schema：定義這個 index 有哪些 field、每個 field 的型別，以及如何被處理與建立索引等屬性。你可以在 document 裡放任何資料，但 mapping 決定哪些欄位可被搜尋、以及它們的資料型別。

型別可以很複雜，例如在 document 裡巢狀物件與陣列、用地理型別、自訂 analyzer，甚至用 embedding 做語意搜尋。這裡不會全部展開，但若你覺得搜尋需要某種特殊處理，Elasticsearch 多半已經有對應功能。

下面是一個 mapping 範例：

```
{
  "properties": {
    "id": { "type": "keyword" },
    "title": { "type": "text" },
    "author": { "type": "text" },
    "price": { "type": "float" },
    "createdAt": { "type": "date" }
  }
}
```

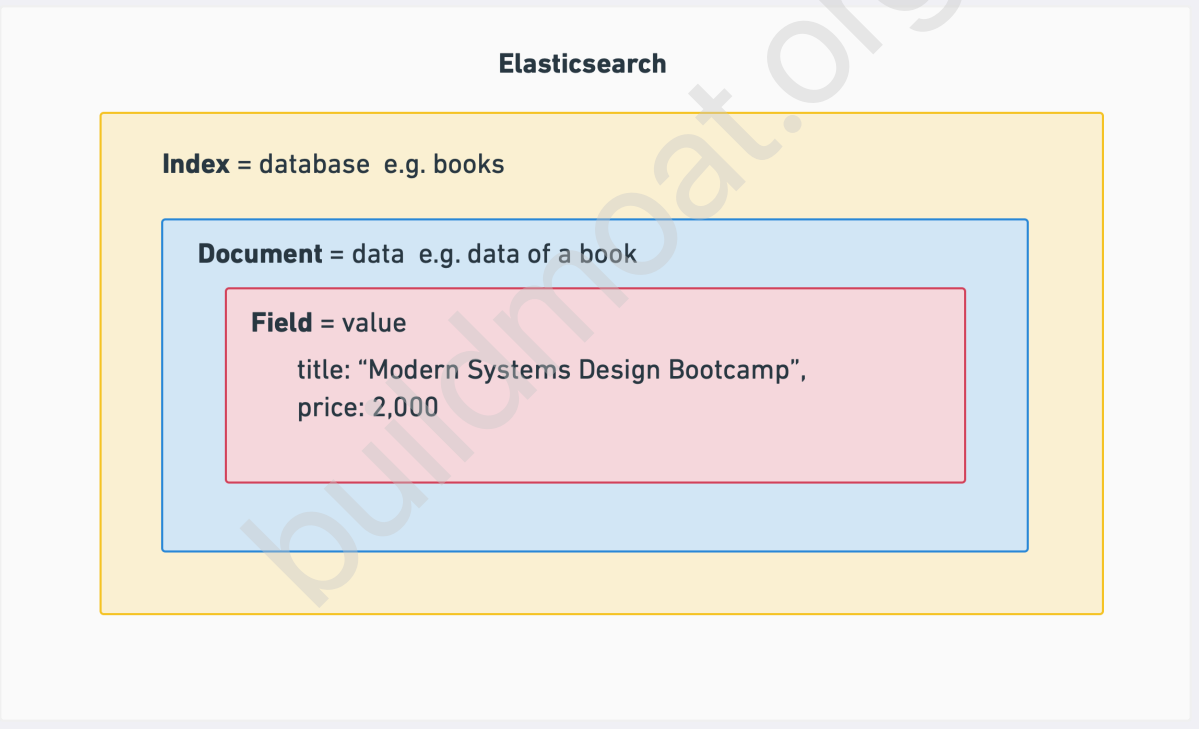
buildmoat.org

Mapping 很重要，因為它告訴 Elasticsearch 如何解讀你存的資料。例如上面把 id 定成 keyword，代表 id 被當成「一整段值」而不是會被分詞的字串，這樣查詢和排序會更有效率（查 id 時比較像 hash table；查 title 時才比較像 reverse index）。

Mapping 也會影響 cluster 的效能：若 mapping 裡放了很多實際上不參與搜尋的欄位，每個 index 的記憶體開銷會變大，可能導致效能問題和成本上升。假設 User 有 10 個欄位，但你只會用其中 2 個做搜尋，若把整個物件都 map 進去，另外 8 個欄位就是在浪費記憶體。這點值得注意，因為很多查詢效能的控制都來自對 mapping 和各種 cluster 參數的調整，後面會再提到。

基本操作

接下來用一連串操作示範：建立 index、寫入資料、執行搜尋，熟悉基本流程。Elasticsearch 提供清楚的 REST API，用 HTTP 就能完成這些事，當然也有不少 GUI 和 client 可用。



buildmoat.org

建立 Index

用一個簡單的 PUT 就能建立 index，可以搭配 dynamic mapping，以及 1 個 shard、1 個 replica（這些參數之後也能改）。

```
// PUT /books
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 1
  }
}
```

設定 Mapping

若不想用 dynamic mapping（例如多數欄位其實不需要被搜尋），可以事先為 index 設定 mapping，讓 Elasticsearch 知道哪些欄位要參與搜尋、以及型別為何。

```
// PUT /books/_mapping
{
  "properties": {
    "title": { "type": "text" },
    "author": { "type": "keyword" },
    "description": { "type": "text" },
    "price": { "type": "float" },
    "publish_date": { "type": "date" },
    "categories": { "type": "keyword" },
    "reviews": {
      "type": "nested",
      "properties": {
        "user": { "type": "keyword" },
        "rating": { "type": "integer" },
        "comment": { "type": "text" }
      }
    }
  }
}
```

這裡預先註冊了書店會用來搜尋的欄位。之後新增 document 時，Elasticsearch 會依這些欄位取值並建立索引，方便搜尋。

buildmoat.org

Mapping 可以很複雜，也會隨資料與需求演進。例如上面的 reviews 是 nested 型別，每一筆評論是巢狀的 document，有自己的一組欄位，和「每筆評論是獨立 document」的扁平結構不同。

要不要把 reviews 巢狀在書的 document 裡，主要看更新與查詢模式，也是面試裡常被問的取捨。若評論很少改、但常被查，巢狀在書裡可能比較省事；否則多半會另建一個 reviews 的 index。這和關聯式資料庫裡的正規化 / 反正規化取捨很像。

新增 Document

有了 index 和 mapping 之後，下一步就是寫入 document，用 HTTP POST 打到 /_doc 即可。

```
// POST /books/_doc
{
  "title": "BuildMoat - Intensive Training Camp (System Desig
  "author": "Terry & Bohr",
  "description": "A comprehensive guide to mastering system d
  "price": 2000,
  "publish_date": "2026-03-15",
  "categories": ["System Design", "Architecture"],
  "reviews": [
    {
      "user": "reader1",
      "rating": 5,
      "comment": "The best system design resource out there."
    },
    {
      "user": "reader2",
      "rating": 4,
      "comment": "Very insightful and practical."
    }
  ]
}
```

如果要再加一本書，那就再發一次 request。

buildmoat.org

```
// POST /books/_doc
{
  "title": "BuildMoat - Intensive Training Camp (Algorithm an
  "author": "Terry & Hank",
  "description": "A comprehensive guide to mastering algorithm
  "price": 2000,
  "publish_date": "2025-08-15",
  "categories": ["Algorithm", "Data Structure"],
  "reviews": [
    {
      "user": "reader3",
      "rating": 5,
      "comment": "A must-read for coding interviews, very prac
    }
  ]
}
```

每次 request 都會回傳 document ID 以及寫入結果的資訊，例如：

```
{
  "_index": "books",
  "_id": "mPFJNYkBq7V9x4qGKPoi",
  "_version": 1,
  "result": "created",
  "_shards": {
    "total": 2,
    "successful": 1,
    "failed": 0
  },
  "_seq_no": 0,
  "_primary_term": 1
}
```

請特別留意 _version。Elasticsearch 用這個欄位來做 document 的原子更新（樂觀鎖）。

buildmoat.org

更新 Document

更新時要在 URL 裡指定 document ID。可以用「整份 document 覆寫」的方式調價，例如：

```
// PUT /books/_doc/mPFJNYkBq7V9x4qGKPoi
{
  "title": "BuildMoat - Intensive Training Camp (System Desig
  "author": "Terry & Bohr",
  "description": "A comprehensive guide to mastering system d
  "price": 999,
  "publish_date": "2026-03-15",
  "categories": ["System Design", "Architecture"],
  "reviews": [
    {
      "user": "reader1",
      "rating": 5,
      "comment": "The best system design resource out there."
    },
    {
      "user": "reader2",
      "rating": 4,
      "comment": "Very insightful and practical."
    }
  ]
}
```

buildmoat.org

這樣做在某些情境合理，但有風險：若同時有另一個 process 在更新同一份 document，你可能會蓋掉別人的修改。

若要避免這種情況，可以用前面的 _version，在 request 裡加上 version 條件，表示「只有當 version 符合時才更新」。下面這個 request 只會在 version 為 1 時更新，否則會回錯：

```
// PUT /books/_doc/mPFJNYkBq7V9x4qGKPoi?version=1
...
```

這樣 client 可以處理衝突並重試，是樂觀並行控制（optimistic concurrency control）的簡單例子。

另外，_update endpoint（注意是 POST）可以只更新部分欄位，不必先讀出整份 document 再寫回：

```
// POST /books/_update/mPFJNYkBq7V9x4qGKPoi
{
  "doc": {
    "price": 999
  }
}
```

對 client 來說這樣做的好處很直覺，但要記得 Elasticsearch 是分散式、非同步、並行的，你的 request 可能被送到多個 node，且順序不一定一致。把更新語意寫清楚，才能確保行為符合預期。

這些 API 設計細節在面試裡討論 API design 時很適合拿出來講，很多公司或面試類型會深入問 API 設計，從這些開源專案學是很好的練習。

搜尋

有了 index 和 document，接下來是怎麼搜。Elasticsearch 的查詢語法接近 SQL 的思維，而且用 JSON 表示，很好組合。

例如搜尋書名包含 “BuildMoat” 的書：

```
// GET /books/_search
{
  "query": {
    "match": {
      "title": "BuildMoat"
    }
  }
}
```

這是 GET 帶 body 嗎？算也不算，Elasticsearch 接受 GET 帶 body，若你前面有很嚴格的 proxy 不允許，可以改把查詢放到 query string 或用 POST endpoint。這裡用這種寫法是為了易讀。

buildmoat.org

也可以加上條件：書名有 “BuildMoat” 且價格小於 1000塊：

```
// GET /books/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "title": "buildmoat" } },
        { "range": { "price": { "lte": 1000 } } }
      ]
    }
  }
}
```

還可以在巢狀的 reviews 裡搜，例如找出有「practical」評價的書：

```
// GET /books/_search
{
  "query": {
    "nested": {
      "path": "reviews",
      "query": {
        "bool": {
          "must": [
            { "match": { "reviews.comment": "practical" } },
            { "range": { "reviews.rating": { "gte": 4 } } }
          ]
        }
      }
    }
  }
}
```

回應可能長這樣：

buildmoat.org

```
{
  "took": 7,
  "timed_out": false,
  "_shards": {
    "total": 5,
    "successful": 5,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 2,
      "relation": "eq"
    },
    "max_score": 3.5,
    "hits": [
      {
        "_index": "books",
        "_type": "_doc",
        "_id": "mPFJNYkBq7V9x4qGKPoi",
        "_score": 3.5,
        "_source": {
          "title": "BuildMoat - Intensive Training Camp (Syste
          "author": "Terry & Bohr",
          "price": 999
        }
      },
      {
        "_index": "books",
        "_type": "_doc",
        "_id": "pQRSTYkBr8W2y5rHLQmk",
        "_score": 2,
        "_source": {
          "title": "BuildMoat - Intensive Training Camp (Algo
          "author": "Terry & Hank",
          "price": 999
        }
      }
```


buildmoat.org

```
    ]
  }
}
```

我們利用 mapping 和一系列條件 / 篩選得到結果。結果裡有符合的 document ID、依相關度算出的分數（_score，稍後會提）、以及若沒特別指定排除的話會帶上 _source（這裡就是書的 JSON）。

排序

篩出有興趣的書之後，要怎麼排序才能把最好的結果排在前面？排序是 Elasticsearch 的重要功能，可以依指定欄位決定結果順序。

基本排序

在 search 的 sort 參數裡指定欄位即可。例如依價格由低到高：

```
// GET /books/_search
{
  "sort": [
    { "price": "asc" }
  ],
  "query": {
    "match_all": {}
  }
}
```

buildmoat.org

也可以多欄位排序，例如先依價格升序後，再依出版日做降序的動作：

```
// GET /books/_search
{
  "sort": [
    { "price": "asc" },
    { "publish_date": "desc" }
  ],
  "query": {
    "match_all": {}
  }
}
```

用 Script 排序

Elasticsearch 支援用自訂 script（Painless）排序，適合依計算值排序。例如依打八折後的價格排序（只是示範，排序結果和原價一樣）：

```
// GET /books/_search
{
  "sort": [
    {
      "_script": {
        "type": "number",
        "script": {
          "source": "doc['price'].value * 0.8"
        },
        "order": "asc"
      }
    }
  ],
  "query": {
    "match_all": {}
  }
}
```

巢狀欄位排序

對 nested 欄位排序要用 nested sort，讓排序用的值來自同一個巢狀物件。例如依最高評論分數排序：

buildmoat.org

```
// GET /books/_search
{
  "sort": [
    {
      "reviews.rating": {
        "order": "desc",
        "mode": "max",
        "nested": {
          "path": "reviews"
        }
      }
    }
  ],
  "query": {
    "match_all": {}
  }
}
```

依相關度排序

若沒有指定 sort，Elasticsearch 會依相關度分數 _score 排序。預設的 scoring 演算法和 TF-IDF（Term Frequency-Inverse Document Frequency）關係密切。若還沒聽過 TF-IDF，花十分鐘了解一下會很值得，在很多情境都會用到。

分頁與 Cursor

定好篩選和排序之後，最後一個問題是怎麼把結果一頁一頁傳給使用者，也就是分頁。Elasticsearch 的分頁讓你可以一次只取一部分結果，通常對應多頁的 UI。除了要決定「每一頁用數字還是用條件表示」，還要考慮是要維持狀態還是每次請求都重新跑一次搜尋。

From/Size 分頁

最簡單的方式是指定 from（起始位置）和 size（每頁筆數），例如：

buildmoat.org

```
// GET /my_index/_search
{
  "from": 0,
  "size": 10,
  "query": {
    "match": {
      "title": "elasticsearch"
    }
  }
}
```

但這種方式在「深分頁」（例如超過一萬筆）時會變得很貴，因為每次請求都要先排序並取出前面所有 document，cluster 要處理的資料量很大。

Search After

對深分頁來說，search_after 比較有效率。它用「上一頁最後一筆的 sort 值」當作下一頁的起點，這樣每一頁只需要取「在上一頁最後一筆之後」的 document，逐步縮小搜尋範圍。例如：

```
// GET /my_index/_search
{
  "size": 10,
  "query": {
    "match": {
      "title": "elasticsearch"
    }
  },
  "sort": [
    {"date": "desc"},
    {"_id": "desc"}
  ],
  "search_after": [1463538857, "654323"]
}
```

search_after 放的是「上一頁最後一筆」的 sort 值。流程是：第一次查詢不帶 search_after；從結果中取最後一筆的 sort 值；把這組值當成下一次查詢的 search_after。上面例子裡 1463538857 是 date 欄位（上一頁最後一筆的時間

buildmoat.org

戳），“654323” 是該 document 的 _id。有了這些值，Elasticsearch 就知道從哪裡開始，即使深分頁也比較省資源。

這樣做的好處是：不會漏掉後來新增的 document（即使兩次請求之間有新資料）；也不會在頁與頁之間重複。代價是 client 要自己記住「上一頁最後一筆的 sort 值」，且無法隨機跳到某一頁，只能一頁一頁往後。若底層資料在分頁過程中被更新或刪除，也可能導致前面幾頁的結果有變動。

Cursor（Point in Time + search_after）

Cursor 提供「有狀態」的分頁，解決「翻頁時底下資料在變」的問題，讓多頁請求之間有一致的視角，但成本比前面兩種分頁方式高。

Elasticsearch 用 Point in Time（PIT）API 搭配 search_after 做 cursor 分頁：

建立 PIT：

```
// POST /my_index/_pit?keep_alive=1m
```

會回傳一個 PIT ID。

在搜尋時使用 PIT：

```
// GET /_search
{
  "size": 10,
  "query": {
    "match": {
      "title": "elasticsearch"
    }
  },
  "pit": {
    "id": "46To...",
    "keep_alive": "1m"
  },
  "sort": [
    {"_score": "desc"},
    {"_id": "asc"}
  ]
}
```

buildmoat.org

要取下一頁時加上 search_after：

```
// GET /_search
{
  "size": 10,
  "query": {
    "match": {
      "title": "elasticsearch"
    }
  },
  "pit": {
    "id": "46To...",
    "keep_alive": "1m"
  },
  "sort": [
    {"_score": "desc"},
    {"_id": "asc"}
  ],
  "search_after": [1.0, "1234"]
}
```

用完記得關閉 PIT：

```
// DELETE /_pit
{
  "id" : "46To..."
}
```

用 PIT 搭配 search_after，在整個分頁過程中會維持一致的資料視角，即使 index 正在被更新也一樣。

底層怎麼運作

了解作為 client 怎麼用 Elasticsearch 之後，下一步就是看這些操作在底層是怎麼實作的。

buildmoat.org

可以把 Elasticsearch 想成「建立在 Apache Lucene 之上的高階協調框架」。Lucene 是高度優化的底層搜尋函式庫，負責搜尋本身；Elasticsearch 負責分散式那層：cluster 協調、API、aggregation、即時能力等。要講的細節很多，我們先看 Elasticsearch cluster 的高階架構，再深入 indexing 和 searching 裡比較關鍵的部分。

Cluster 架構

Node 類型

Elasticsearch 是分散式搜尋引擎，啟動一個 cluster 其實是啟動多個 node。Node 有五種型別，在啟動 instance 時設定。

Master Node 負責協調整個 cluster，是唯一能執行 cluster 級操作（例如增刪 node、建立或刪除 index）的 node，可以想成「管理員」。

Data Node 負責存資料，你的資料實際存在這裡，大型 cluster 裡會有很多個。

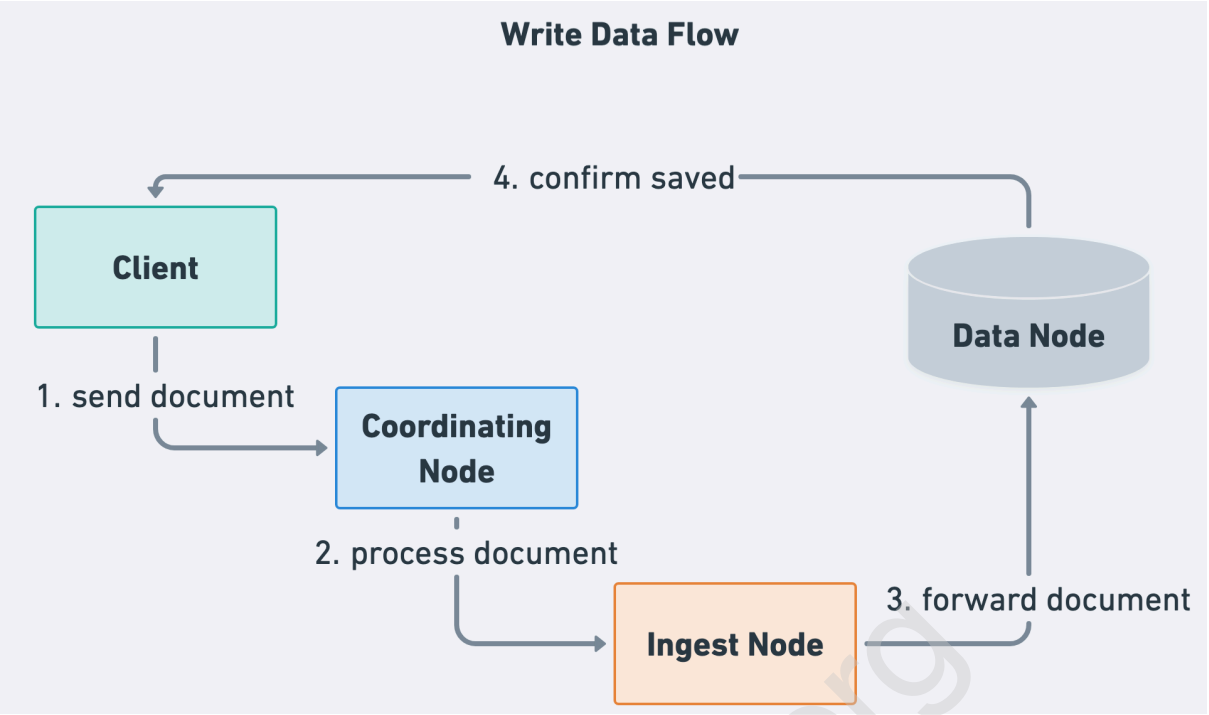
Coordinating Node 負責協調搜尋請求，接收 client 的搜尋請求並轉發到對應的 node，是 cluster 對外的入口。

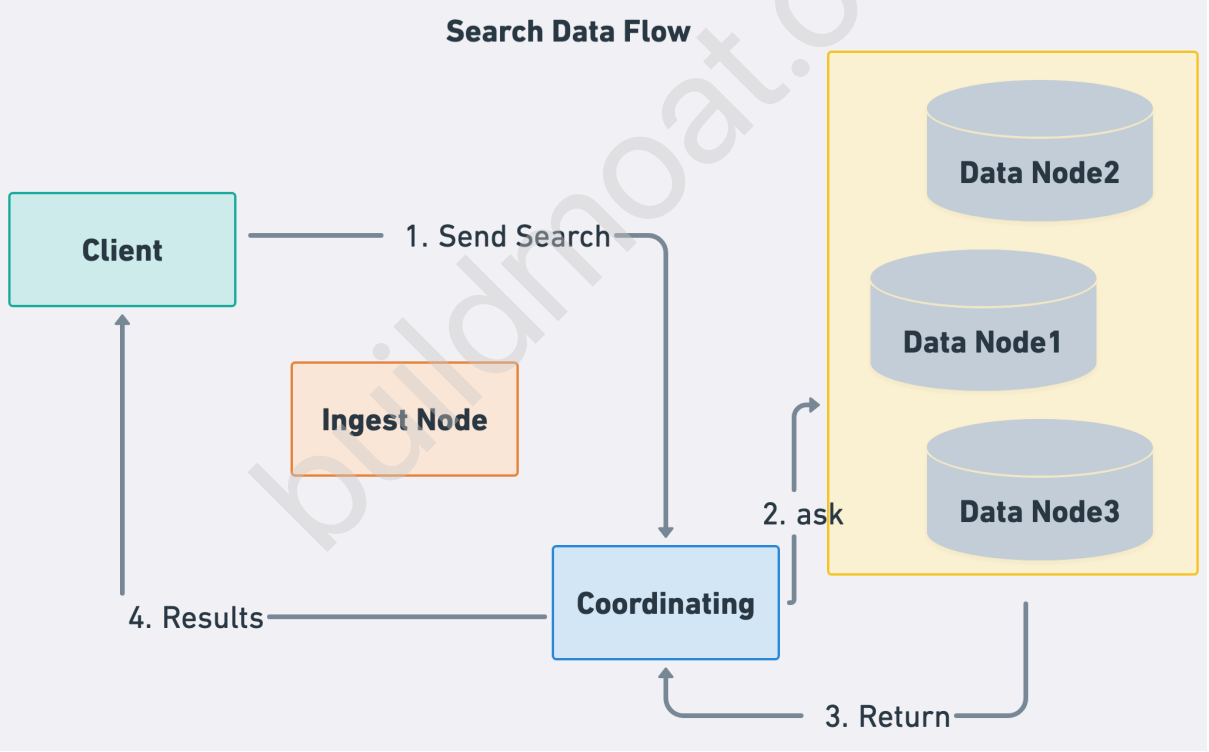
Ingest Node 負責資料攝入，在這裡做轉換與前置處理再送進 indexing。

Machine Learning Node 負責機器學習相關工作。

這些 node 的協作方式很直觀：寫入和查詢都先經由 Coordinating node 接收。寫入時，Coordinating node 會先判斷是否需要經過 Ingest node 做前置處理（如有設定 ingest pipeline），處理完後再由 Coordinating node 根據 routing（通常是 `hash(_id) % number_of_shards`）決定寫入哪個 primary shard，轉發到對應的 Data node。查詢時，Coordinating node 則將請求分發到相關的 shard，再合併結果回傳給 client。

bullmoat.org





buildmoat.org

同一個 Elasticsearch instance 可以同時扮演多種角色，由設定決定。例如可以同時是 master-eligible 和 coordinating node。在較複雜的部署裡，可能會為不同類型用不同主機（例如 ingest 主機偏 CPU、data 主機偏磁碟 I/O 或記憶體）。

每種 node 型別還有細分。例如 Data node 可以分成 hot、warm、cold、frozen，依資料被查詢的機率（例如新舊）以及是否會變動來區分。

Cluster 啟動時會指定一組 seed nodes（master-eligible），它們會跑 leader election 選出一個 master。同一時間只應有一個 active master，其他 master-eligible 處於 standby。

Ingest 和 Coordinating node 很重要，但「搜尋」真正發生在 Data node，所以我們從這裡往下講。

Data Node

Data node 的主要工作是存 document 並讓它們能被快速搜尋。做法是把原始的 _source 資料（搜尋結果裡看過的那個）和 Lucene 用來搜尋的 index 分開存，可以想成另外有一份「文件庫」。

請求分兩階段：先「query」階段，用優化過的 index 資料結構找出符合的 document；再「fetch」階段，依需求從各 node 拉出這些 document 的內容（例如 _source）。理想的查詢是盡量不碰 _source，有時會把常用欄位放進 index 以 included fields 方式提供。

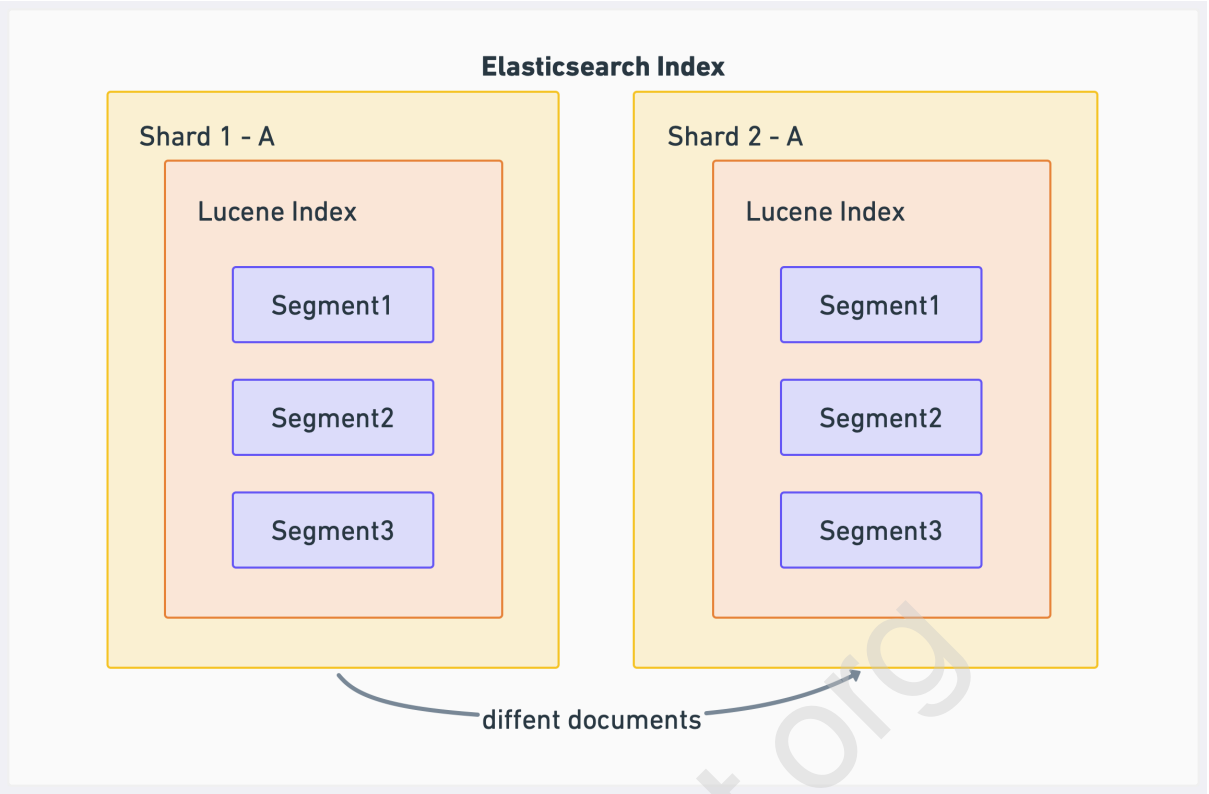
Data node 上面放的是我們前面說的 index，index 由 shard 和 replica 組成；shard 裡面是 Lucene index，Lucene index 又由 Lucene segment 組成。

Shard 讓 Elasticsearch 能把資料（以及對應的 index）切開分散到多台主機，把 document 和 index 結構分布到 cluster 的多個 node，從而提升效能與擴展性。搜尋會並行跑在所有相關 shard 上，結果由 coordinating node 合併與排序。一般流程是：查詢先到 coordinating node，再被分發到對應的 shard。

Replica 是某個 shard 的完整拷貝。Elasticsearch 可以為 index 的每個 shard 建立一或多個副本（replica shard）。Replica 有兩個主要用途：高可用與提高吞吐。若一個 shard 能處理 X TPS，有 Y 個 replica 時（其他條件不變）理論上可以到 X * Y TPS。Coordinating node 可以把搜尋請求分散到所有可用的 shard 副本（primary 和 replica），達到負載平衡。

最後，Elasticsearch 的 shard 和 Lucene index 是一對一。前面說過 Lucene 是 Elasticsearch 底層的搜尋函式庫，很多 Elasticsearch 對 shard 做的操作（merge、split、refresh、search）其實都是對底下 Lucene index 的代理。到這裡可以簡化理解：Elasticsearch 就是在一大群 Lucene index 之上加上可用性與擴展性。

kuindroat.org



Lucene Segment 的 CRUD

Lucene index 由 segment 組成，segment 是搜尋引擎的基本單位，是「不可變的」索引資料容器。先記住「不可變」這件事。我們不是要能更新、新增、刪除 document 嗎？Lucene 的做法是：寫入先批次累積，再建 segment。插入 document 時不會立刻寫進既有的 index，而是先放進一個 segment；累積一批後，建好一個 segment 再 flush 到磁碟。Segment 太多時可以 merge：把多個 segment 合併成一個新 segment，再刪掉舊的。

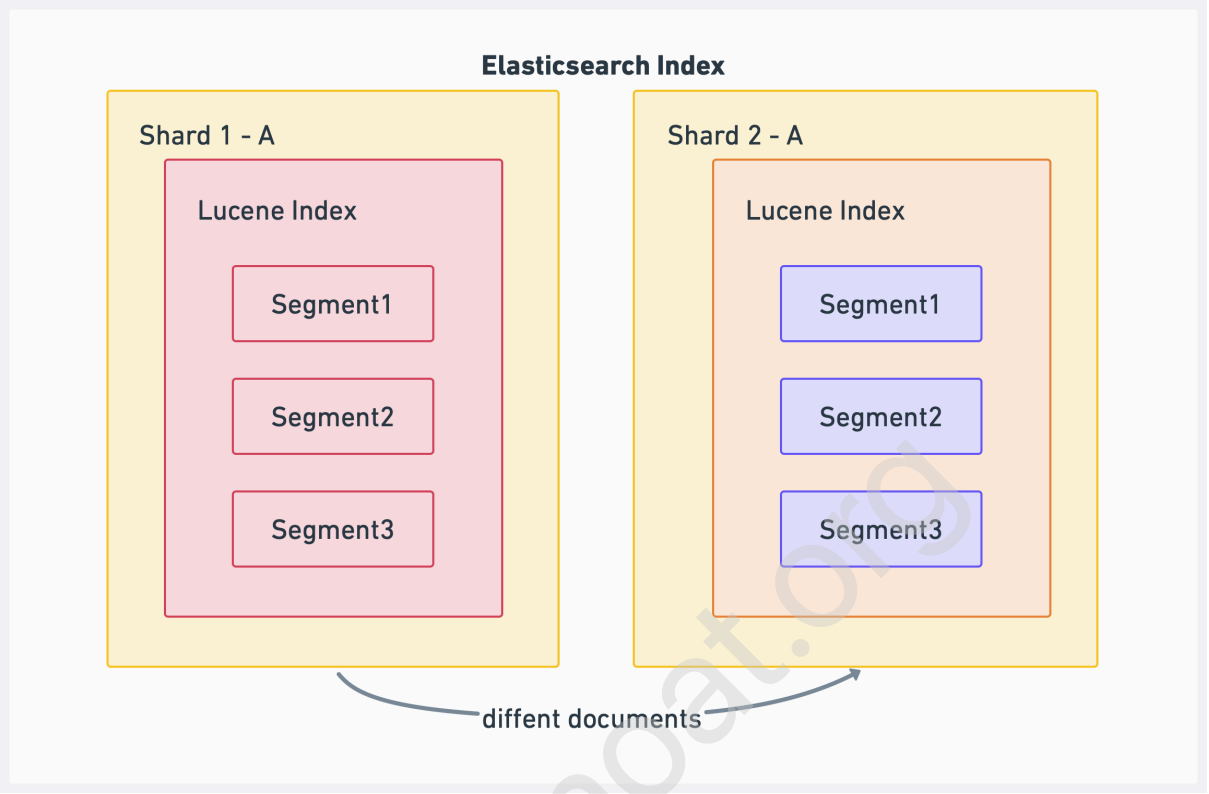
刪除比較特別：每個 segment 會維護一組「已刪除」的 ID。查詢時若碰到已刪除的 document，就當作不存在，但資料其實還留在 segment 裡；在 merge 時才會真正清掉。更新則是：不直接改 segment，而是對舊 document 做 soft delete，再插入一筆帶新資料的 document，舊的那筆在之後的 segment merge 時一併清理。所以刪除很快，但在 merge 前會一直帶著效能包袱，理想上不要頻繁更新。

更新比插入更耗效能，因為還要處理 soft delete 的簿記，這也是為什麼 Elasticsearch 不適合「寫入與更新非常頻繁」的資料。

這種不可變設計給 Lucene 帶來不少好處：寫入快（新 document 寫進新 segment，不用改舊的）、快取簡單（segment 不變，可以安心 cache 在記憶體或 SSD）、並行單純（讀取不用擔心資料查一半被改）、恢復容易、壓縮效果好、搜尋可以針對不可變結構做優化。代價是需要定期做 segment merge，以及 merge 前暫時會多佔一些空

buildmoat.org

間。Elasticsearch 和 Lucene 有一整套機制在平衡這些取捨。這類「用不可變換取效能與簡化」的設計，在偏資料基礎設施的系統設計面試裡很常被討論，也可以套用到你設計的其他系統。



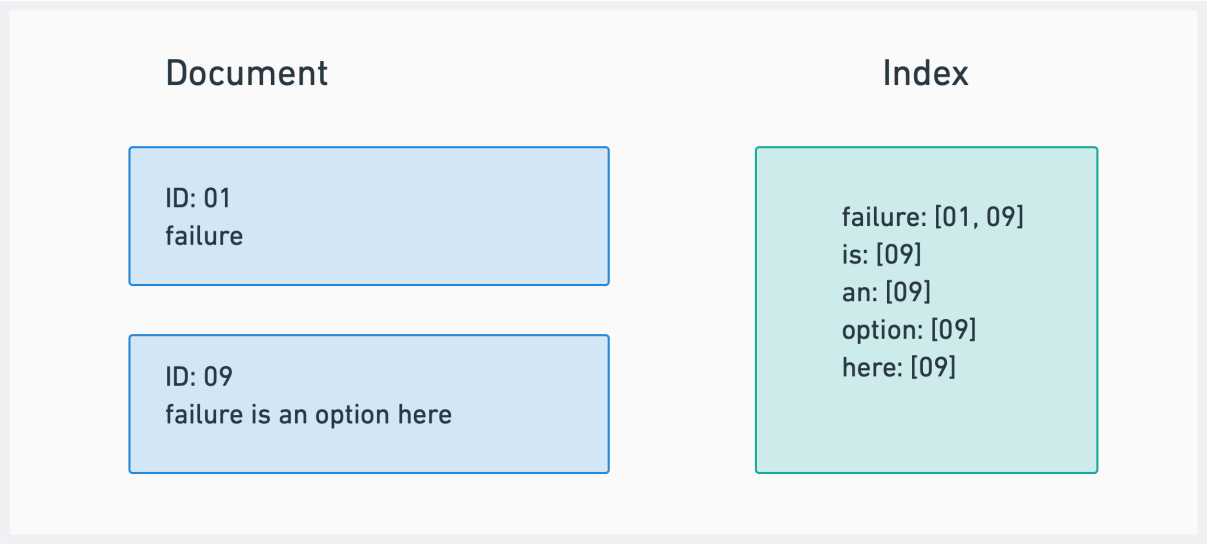
Lucene Segment 的結構

Segment 不只是裝 document 的容器，裡面還有針對搜尋優化過的資料結構，其中最重要的兩個是 inverted index 和 doc values。

Inverted Index

若 Lucene 是 Elasticsearch 的心臟，inverted index 就是 Lucene 的心臟。本質上想「找東西很快」有兩條路：一是把資料照「你怎麼取用」來組織（例如要精確查就用 hash table O(1)，不要掃整張表 O(n)）；二是複製一份資料，把那份照 (1) 的方式組織。假設有十億本書，其中少數書名包含 “lazy”，我們想寫程式快速找出所有書名含 “lazy” 的書，該怎麼做？

buildmoat.org



Inverted index 是一種「從內容（例如詞或數字）對應到它在文件 / document 裡位置」的資料結構，這也是 Elasticsearch 關鍵字搜尋能很快的原因。它列出所有在 document 裡出現過的不重複詞，並記錄每個詞出現在哪些 document（例如字串 “lazy” 對應到 document #12 和 #53）。這樣就不用掃過每一份 document，只要查 inverted index 就能在常數時間找到包含 “lazy” 的 document，等於用「複製＋巧妙組織」把 O(n) 掃描變成 O(1) 查表。

Doc Values

那如果我們想依價格排序呢？這時用 doc values。Document 裡有作者、書名等很多欄位，但排序時我們只需要「所有符合結果的價格」。這在關聯式資料庫這種以 row 為主的儲存裡很常見：即使只用到一欄，往往也要讀整行再取那一欄。像 Spark 或 AWS Redshift 這類分析型系統的秘訣之一，就是用欄式（columnar）儲存，把一欄的資料連續放在一起，查一欄就讀一段連續記憶體。Doc values 就是對「單一欄位、在整個 segment 所有 document 上」做這種欄式、連續的表示。Inverted index 告訴我們「哪些 document 符合」，doc values 提供我們做最後排序（或 aggregation）時需要的欄位值。

Coordinating Node

Elasticsearch 是分散式系統，Coordinating node 負責接收 end client 的請求，並在 cluster 裡協調執行。它是使用者請求的入口，負責解析查詢、決定哪些 node 要參與、以及把結果回傳給使用者。

執行過程中很重要的一步是 query planning。Query planner 會決定「用什麼方式執行搜尋最省」。Coordinating node 解析完查詢後，query planner 會評估怎麼取回符合的 document，包括要不要用 inverted index、查詢各部分的最佳執行順序、以及多個 node 的結果要怎麼合併。

kuimoat.org

用一個簡單例子說明「順序」的影響。你在幾百萬份 document 裡搜 “bill nye”。Inverted index 裡 “bill” 有幾百萬筆，“nye” 只有幾百筆。你可以：先建 “nye” 的 document 集合，再掃 “bill” 的找交集；或反過來先 “bill” 再 “nye”；或只載 “nye” 的 document 再字串比對 “bill nye”；或只載 “bill” 的再比對……選不同策略，效能可能差好幾個數量級。Elasticsearch 的 query planner 會根據欄位型別、詞的分布、document 長度等統計資訊選策略，盡量縮短回傳結果的時間。在偏基礎設施的系統設計面試裡，這類「靠統計與一層間接，讓系統依資料動態調整」的設計很常見，也是資料庫系統之所以強大的原因之一。

在面試中怎麼用 Elasticsearch

很多系統設計題目會自然用到 Elasticsearch，只要題目涉及複雜搜尋，通常都是好候選。實務上 Elasticsearch 也常透過 Change Data Capture（CDC）接到權威資料來源（例如 Postgres 或 DynamoDB）。

使用 Elasticsearch 時要注意的事

用 Elasticsearch 時可以記住幾點，面試時也容易講得清楚：

一般不建議把 Elasticsearch 當成唯一的資料庫。它首先是搜尋引擎，雖然功能很強，但設計目標不是取代傳統資料庫。早期版本在一致性與持久性上出過不少問題，CouchDB 遇到過的不少問題在 Elasticsearch 也出現過。若資料必須持久、一致，建議放在別的地方，Elasticsearch 當搜尋層。

Elasticsearch 偏向讀多寫少。若是寫入很重的系統，要考慮其他方案或加一層 write buffer。雖然可以方便地加「按讚數」「曝光數」這類欄位，但這類高頻更新會讓 Elasticsearch 很吃力。

要考慮 eventual consistency。搜尋結果可能落後，有時落後不少。若你的情境無法接受，可能要找其他方案。

Elasticsearch 不是關聯式資料庫，資料要盡量反正規化，讓搜尋能用一兩次查詢就拿到結果，這可能代表寫入端要多做一點轉換與組裝。

不是所有搜尋問題都需要 Elasticsearch。若資料量小（例如不到十萬筆）或很少變動，可能有更簡單、更快的做法。先看用主要資料庫做簡單查詢是否夠用，不夠再考慮 Elasticsearch。

要確保 Elasticsearch 和底層資料保持同步。同步失敗會造成資料漂移，是使用 Elasticsearch 時常見的 bug 來源。

Elasticsearch 很強，但不是銀彈。要能說明為什麼選它、以及它的限制與適用場景。

pulinoat.org

從 Elasticsearch 可以學到的設計觀念

即使不用 Elasticsearch，從它的設計裡也能抽出一些通用原則，用在高效能基礎設施的設計上：

在合適的層級使用不可變性能帶來很大效益。資料保持不變，就更容易快取、壓縮與優化，也比較不需要處理可變資料的同步與一致性難題。

把「查詢執行」和「資料儲存」分開，可以各自優化。Elasticsearch 的 Data node 和 Coordinating node 各司其職，就是很好的例子。

索引策略會直接影響搜尋效能。Inverted index 讓全文搜尋快，doc values 讓排序與 aggregation 有效率。設計需要快速檢索的系統時，要根據常見查詢模式來設計資料與索引結構。

分散式系統能提供擴展性與容錯，但也帶來複雜度。Elasticsearch 的 cluster 架構能處理大量資料與高查詢負載，但必須認真考慮資料一致性與網路分區。設計分散式系統時，要權衡 CAP 中的 consistency、availability、partition tolerance。

高效能資料結構非常重要。Elasticsearch 在 inverted index 裡使用 skip list、finite state transducer 等結構，說明針對使用情境選擇或設計資料結構能大幅提升效能。設計或選型時要根據資料的存取模式來考慮。

buildmoat.org